

Post-mortem отладка приложений: вообще и в Тензоре

Каждый разработчик мечтает, чтобы в его приложении не было ошибок и оно приносило максимальную пользу своим потребителям, вызывая лишь приятные эмоции. Однако суровая реальность нашей жизни такова, что в любом ПО ошибки все-таки присутствуют в том или ином количестве. Причем присутствуют они как в нашем собственном коде, так и в коде тех библиотек, на которые наш код опирается (даже если эти библиотеки очень популярны и используются во множестве других проектов), и даже в самой операционной системе. Все эти ошибки имеют разные типы и разную степень деструктивности последствий. Если оставить за скобками (на самом деле весьма неприятные) логические и архитектурные просчеты, в большинстве своем мы будем иметь дело с ошибками, которые выражаются через выброс исключения того или иного рода при выполнении какой-либо операции в системе. Большинство из этих исключений может быть обработано разработчиком таким образом, что исполнение программы будет продолжено (что особенно актуально в системах массового обслуживания, к которым относятся все наши серверные решения). Однако есть класс ошибок, при возникновении которых выполнение программы по тем или иным причинам не может быть продолжено и приложение "падает".

Во многих случаях такое "падение" сопровождается созданием на уровне ОС так называемого **core-дампа** приложения, который содержит слепок памяти процесса в момент сбоя, и, как предполагается, сможет помочь разработчику выявить и исправить причину возникновения этого самого сбоя. Процесс исследования ошибок при помощи core-дампов называется **post-mortem** (т.е. "посмертной", в противоположность отладке "живого" и работающего процесса) отладкой, или, менее официально, "**курением дампов**". На самом деле, core-дампы являются крайне удобным инструментом, предоставляющим множество полезной информации о "последних мгновениях" жизни процесса, и их очень не хватает, скажем, при разработке под мобильные платформы. Однако, несмотря на повсеместную распространенность данного инструмента отладки и огромное изобилие tutorиалов на просторах интернета, по неизвестным автору причинам в широких народных массах разработчиков Тензора core-дампы приобрели дурную славу каких-то полумагических артефактов, понимание которых доступно лишь немногим разработчикам. В данной статье я попробую осветить некоторые аспекты темы "курения дампов", дабы развеять вышеуказанный магический ореол и сделать дампы просто еще одним (и, как уже упоминалось выше, весьма удобным) инструментом в арсенале каждого разработчика нашей компании.

Что вообще такое этот core-дамп и когда он появляется?

Как уже было упомянуто, core-дамп - это не что иное, как слепок памяти и регистров приложения в момент принятия операционной системой, возможно, на основе запроса от кода самого приложения, решения о его (приложения) аварийном завершении. Предназначен он исключительно для разработчиков упавшего приложения (ну или каких-то добровольцев из комьюнити). Цель записи core-файла - предоставление разработчику максимума информации о состоянии процесса в момент сбоя. Как вы понимаете, как правило (по крайней мере количественно) программы "падают" не у разработчиков, а у конечных пользователей. Также, как вы понимаете, в памяти приложения может содержаться какая-либо конфиденциальная для пользователя информация. Кроме того, core-дамп типичного процесса занимает достаточно много места на диске. По вышеуказанным причинам по умолчанию запись дампов отключена во всех операционных системах, под которые наша компания ведет разработку. Однако при разработке серверных решений и работе в приближенных к "тепличным" условиях ЦОД-а мы можем позволить себе такую роскошь, как активацию записи дампов (это сделано в базовых шаблонах наших виртуальных машин). Более того, у нас есть специализированная инфраструктура, которая эти самые дампы не только записывает так, как нам надо и тогда, когда нам надо, но и обогащает их дополнительной ценной информацией, а впоследствии производит первичный разбор. Также ввиду имеющейся возможности в ОС Windows заказывать запись дампа текущего приложения из его собственного кода, мы можем генерировать дампы и для наших десктопных решений под эту ОС, не внося изменений в системные настройки.

Дамп записывается при возникновении ошибки, после которой выполнение процесса не может быть продолжено. Среди типовых причин можно перечислить следующие:

- **SIGSEGV**, он же **Segmentation fault**, он же **Access Violation**, он же "**ошибка доступа**", он же "**проезд по памяти**".

Это ситуация, при которой код приложения пытается обратиться по виртуальному адресу, который отсутствует в виртуальном адресном пространстве исполняемого процесса, либо записать данные в read-only область памяти. В этом случае ОС четко понимает, что в программе баг.

- **SIGABORT** и его аналоги.

Аварийное завершение приложения по явному запросу из его кода (под "его" кодом здесь понимается как собственный код самого приложения, так и код рантайма того ЯП, на котором оно написано). Также SIGABORT (по

крайне мере в Linux) можно послать процессу явно, например, через команду `kill` или используя утилиту `htop`. Применительно к ЯП C++ завершение процесса с таким сигналом возможно, например, при выбросе исключения из `pthread_t`-функции (например, деструктора) или при явном вызове `std::terminate()`.

- **SIGBUS** (Linux-специфично).

"Дефолт" ОС по ранее выданным обязательствам относительно памяти. Возникает при применении `overcommit`-а памяти (который мы активно применяем). Почитать про "ужасающие" особенности управления памятью в современных ОС можно, например, [тут](#).

Также на практике встречаются `core`-дампы с другими сигналами (например, `SIGILL`), но это большая редкость.

Кроме того, как я уже упоминал ранее, `core`-дамп можно сгенерировать вручную, причем без "убийства" того процесса, с которого мы снимаем дамп.

В ОС Windows это делается при помощи диспетчера задач (для 32-х битных приложений для этой цели необходимо использовать 32-битный диспетчер задач!), в Linux - при помощи утилиты [gcore](#).

А нельзя ли не падать?

Достаточно частый вопрос, который можно услышать от коллег, когда им сообщают, что запрос с их методом упал звучит примерно так: "Неужели нельзя как-нибудь так сделать, чтобы процесс не падал?". Ну и вдогонку к нему обычно спрашивают: "А где же логи/статистика/whatever из точки падения".

Короткие ответы на эти вопросы звучат так: не падать нельзя, что-то сильно осмысленное в точке падения сделать тоже не выйдет. Но на самом деле, как это обычно и бывает в жизни, короткий ответ является не совсем правильным. Если очень сильно хочется, то можно и "побороться за свою жизнь" и попробовать что-то сделать в точке сбоя.

Теперь обо всем по порядку. Все известные автору операционные системы позволяют так или иначе обработать приводящие к сбою сигналы ОС. В Windows вообще реализована целая система так называемых структурных исключений. Т.е. в том же Linux мы можем назначить обработчик, например, на сигнал 11 (он же `SIGSEGV`) и попробовать что-то в нем поделаться. Но есть куча нюансов:

- Обработчик будет вызван на случайном потоке и в каком-то не сильно адекватном с точки зрения вашего стека вызовов месте.
- Внутри обработчика безопасно можно звать только весьма и весьма ограниченный список API операционной системы. Например, выделять память через `malloc()` там нельзя. И освободить через `free()` тоже нельзя. И "обычное" исключение кинуть тоже нельзя.

Можно, конечно, попробовать вернуть из обработчика сигнала код "забей, все в порядке" и тем самым попробовать продолжить исполнение. Но тут нас может ждать целый ворох неприятностей:

- Обычно после "подавления" одного сигнала мы получаем целую пачку аналогичных. Иногда этот поток может стать бесконечным.
- Тот код, работа которого привела к генерации сигнала, по сути не исполнит заказанную операцию, но при этом никак об этом не узнает.

Например, мы просили записать N байт в разделяемую память, получили (скорее всего очень много раз) `SIGBUS`, "задавили" их все обработчиком и продолжили работу как ни в чем не бывало. Что в итоге? Операция не исполнена, но код идет по "успешной" ветке. Заканчивается все это, как правило, либо зависанием, либо бесконечным ворохом "производных" сигналов (и тоже зависание, но с поеданием процессора), либо повреждением данных программы.

Все рассмотренные исходы с точки зрения общего здоровья системы гораздо хуже, чем аварийное завершение.

Что нужно для успешной отладки дампа?

Для того чтобы успешно "вскрыть" дамп, нам, в первую очередь, нужен отладчик.

Для Windows это может быть CDB, WinDbg или встроенный отладчик среды Microsoft Visual Studio (все они умеют работать с дампами в формате *.pdb).

Для Linux - GDB достаточно свежей версии (на момент написания данной статьи достаточно свежий GDB в дистрибутиве CentOS 7 можно найти в пакете `devtoolset-9-gdb`). Отдельно обращаю внимание, что компилятор, линковщик и любые другие компоненты элементы тулчейна для разработки на C++ не требуются - нужен только отладчик.

Отладчику же для успешного анализа нужны три компонента:

- Собственно дамп
- Бинарные файлы упавшего процесса (в идеале - все)
- Отладочные символы упавшего процесса

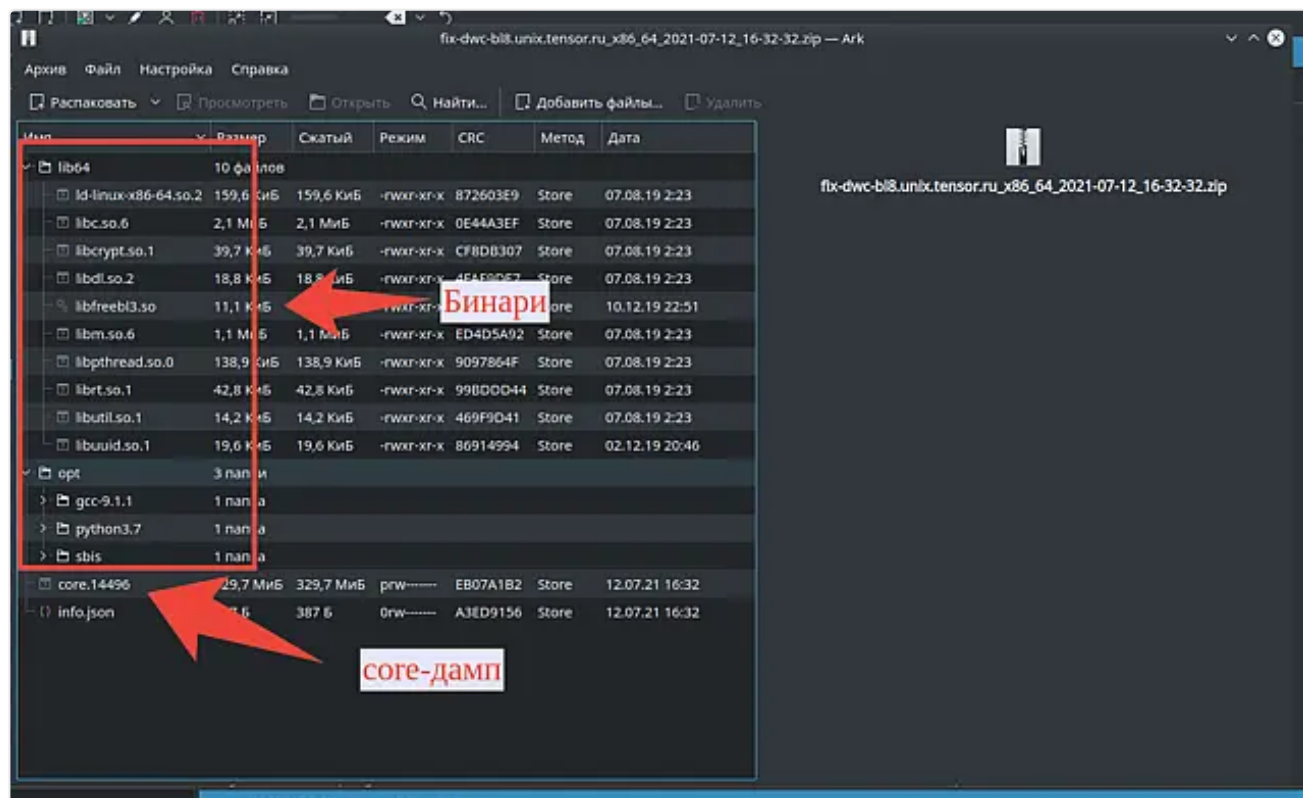
Человеку дополнительно обычно требуется исходный код приложения.

Если вы занимаетесь анализом дампа, то сам дамп у вас наверняка есть. Соответственно, осталось определиться с тем, где взять остальные составляющие успеха. Процесс добычи необходимых компонентов на разных платформах

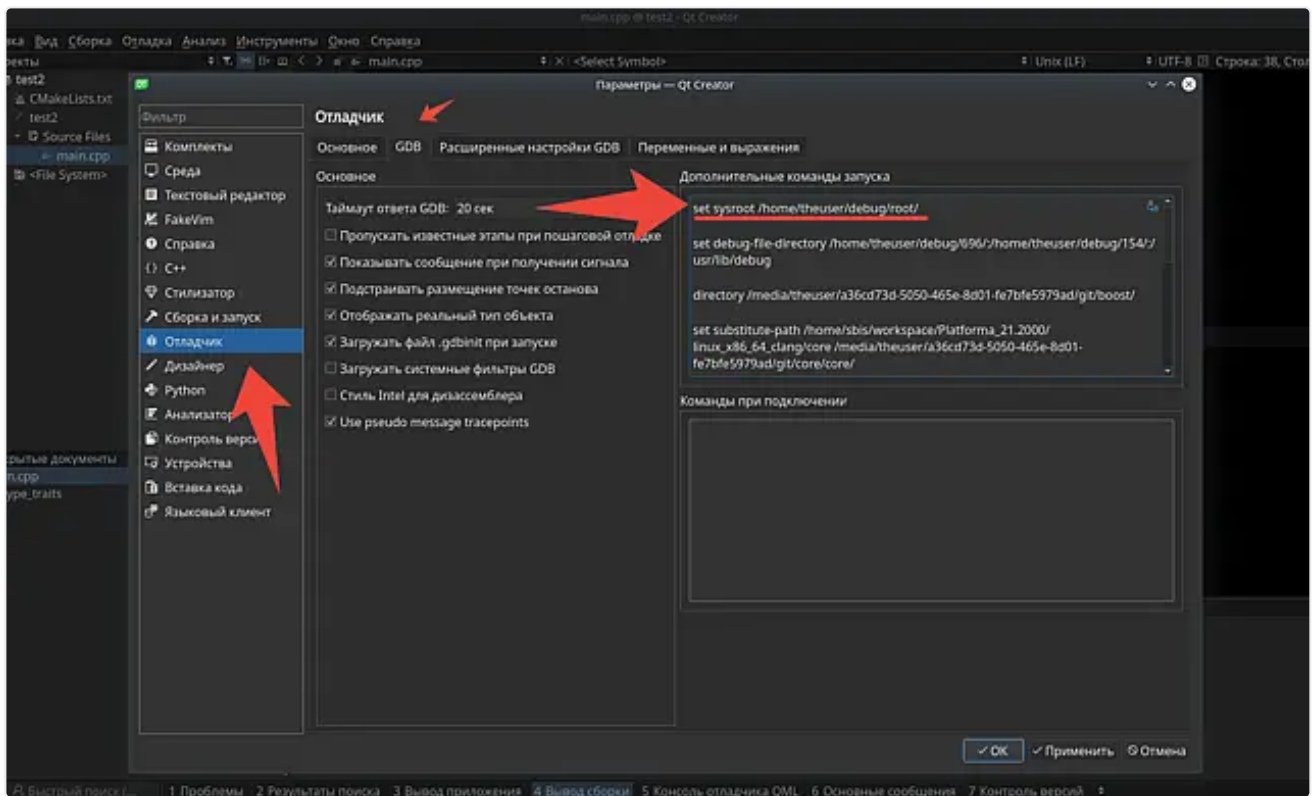
существенно отличается. Как правило, для успешного анализа добывать все binary и символы не требуется. Однако большая часть все-таки должна присутствовать и особенно это касается основополагающих библиотек рантайма C/C++ и платформы. Далее рассмотрим каждый компонент более подробно.

Где взять binary в Linux?

Если речь идет о Linux, то в типовом сценарии (когда дамп записан Тензорской утилитой сбора дампов), добывать ничего не требуется - все binary уже заботливо упакованы в архив с дампом. Остается этот самый архив либо самостоятельно забрать с тестовых стендов dev-symbols.unix.tensor.ru (на компьютерах разработчиков примонтировано в /mnt/dumps), либо запросить у службы эксплуатации, если дамп поступил с продуктива. Что еще более приятно, внутри архива binary располагаются ровно так, как они были расположены на том сервере, где дамп был сгенерирован.



Это позволяет после разархивирования дампа сразу же указать их месторасположение отладчику GDB при помощи команды `set sysroot`. Если анализ проводится в IDE Qt Creator, то перед открытием дампа удобно бывает вписать соответствующую команду GDB в поле "Дополнительные команды запуска" на странице настроек отладчика. После завершения анализа эту инструкцию надо не забыть удалить/закомментировать, иначе вы более не сможете отлаживать собранные локально приложения.



Если же дампы были сгенерированы самостоятельно (например, при помощи утилиты `gcore`), то binary вам также придется добывать самостоятельно. Как правило, для этого достаточно забрать себе содержимое каталога приложения (если речь идет о серверных решениях, то это, как правило, подкаталог каталога `/opt/sbis`), а также некоторые binary из `/bin64` (их точный список зависит от модульного состава приложения). Из обязательных - `libc` и `libpthread` (учтите, что как правило, используется один из симлинков на эти библиотеки и нужно брать именно его). После извлечения с сервера указанных каталогов, следует все файлы `.so` перенести в выделенный каталог и далее уже этот каталог указать GDB как место поиска бинарных файлов при помощи инструкции `set solib-search-path`. Если анализ проводится в IDE Qt Creator, то перед открытием дампа удобно бывает вписать соответствующую команду GDB в поле "Дополнительные команды запуска" на странице настроек отладчика.

Где взять binary в Windows?

Если же речь идет о Windows, то, как правило, binary можно взять с серверов символов: наших корпоративных и серверов Microsoft. Для этого адреса этих серверов необходимо сообщить отладчику. Способ указания сервера символов отличается для разных отладчиков (его можно посмотреть в онлайн-справке). Для используемого нами в комбинации с IDE Qt Creator отладчика CDB это выглядит примерно так:

```
srv\*C:\\symbolcache\\*https://msdl.microsoft.com/download/symbols
```

Здесь первая часть - расположение локального кэша символов (используется для ускорения последующих запросов), вторая часть - собственно адрес сервера символов (в данном случае публичного сервера символов Microsoft).

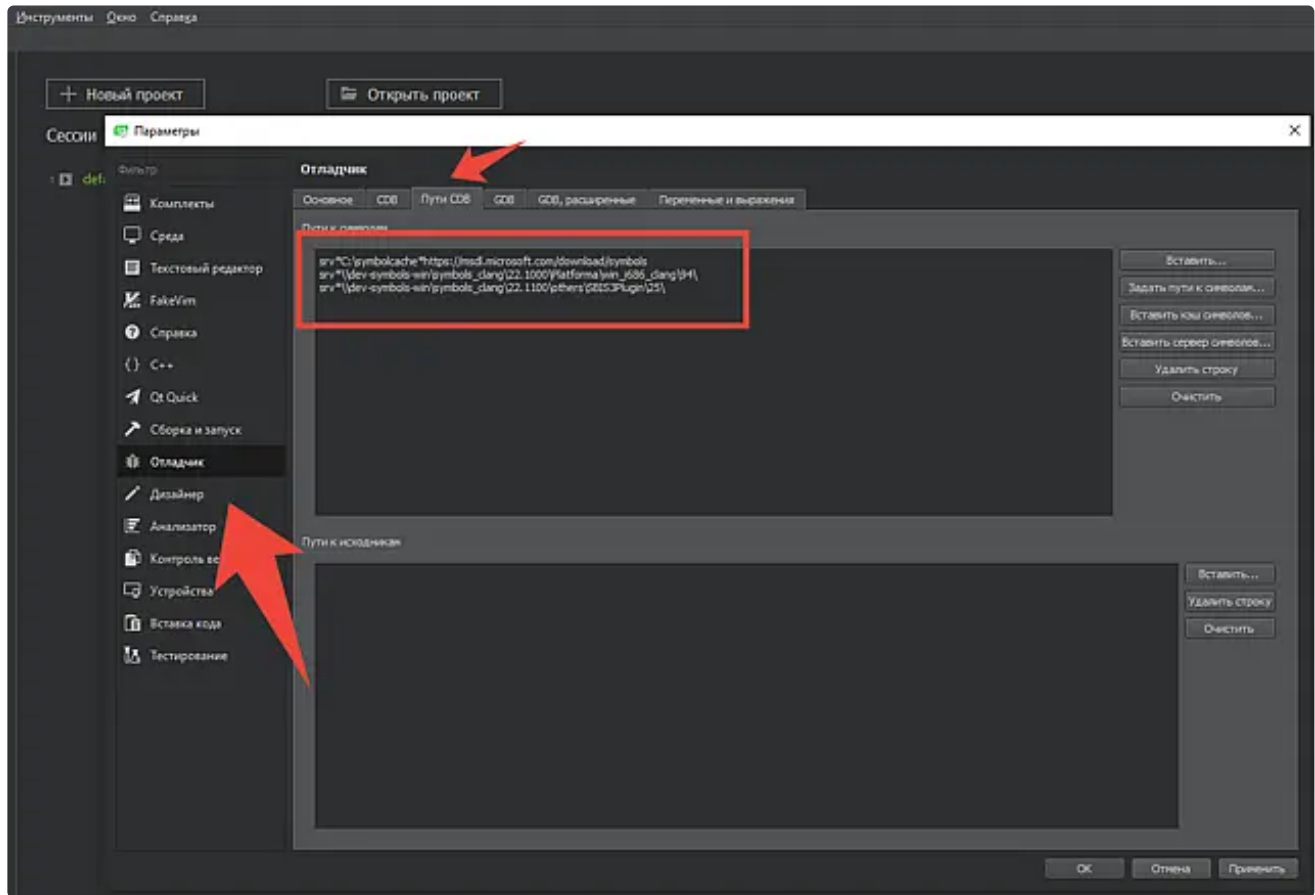
В Тензоре нет единого сервера символов. Каждая сборка платформы и каждая сборка прикладного решения под Windows продуцируют свой маленький сервер символов, соответствующий данной конкретной сборке. Так сделано, чтобы можно было легко отслеживать актуальность того или иного набора бинарников и своевременно очищать диск, когда в данной конкретной сборке более нет необходимости. Так как любое конечное прикладное решение состоит из двух частей: SDK платформы и собственного кода, то для отладки дампа потребуются указать два сервера символов: один для платформы и один для приложения. Все серверы символов на момент написания данной статьи можно найти на сетевой шаре [\\dev-symbols-win\\symbols_clang](https://dev-symbols-win\\symbols_clang). Сама эта шара имеет трехуровневую иерархию: версия → продукт → сборка продукта.

В конечном итоге, в типовом сценарии мы должны дать отладчику (если речь о CDB) примерно следующие инструкции:

```
1  srv\*${ЛокальныйКэшСимволов}\\*https://msdl.microsoft.com/download/symbols
2  srv\*\\\\dev-symbols-
   win\\symbols_clang\\${ВерсияSDKПлатформы}\\Platforma\\win_i686_clang\\$
   {НомерСборкиПлатформы}\\
3  srv\*\\\\dev-symbols-
```

```
win\\symbols_clang\\${ВерсияКонечногоРешения}\\${КаталогКонечногоРешения}\\${НомерСборкиКонечногоРешения}\\
```

Пример настройки IDE Qt Creator представлен на скрине ниже.



СИМВОЛЫ

Для успешного анализа дампа (да и вообще отладки чего-либо, написанного на компилируемых языках программирования) отладчику требуются так называемые отладочные символы. Это данные в специальном формате, описывающие связь между итоговым скомпилированным бинарником и исходным кодом программы.

Формат отладочных символов является платформоспецифичным. На Unix-подобных ОС (в т.ч. Linux) как правило применяется открытый формат [DWARF](#), на Windows применяется проприетарный формат [PDB](#).

Отладочные символы могут быть встроенными в бинарные файлы приложения (так происходит, например, при локальной сборке с настройками по умолчанию на Linux), а могут быть отцепленными (для DWARF отцепление выполняется явно при помощи утилиты strip, PDB всегда является отцепленным). Если применяются отцепленные символы, то в бинарном файле в том или ином виде остается некая ссылка, позволяющая верифицировать, что переданный файл с отладочными символами действительно соответствует данному конкретному бинарнику.

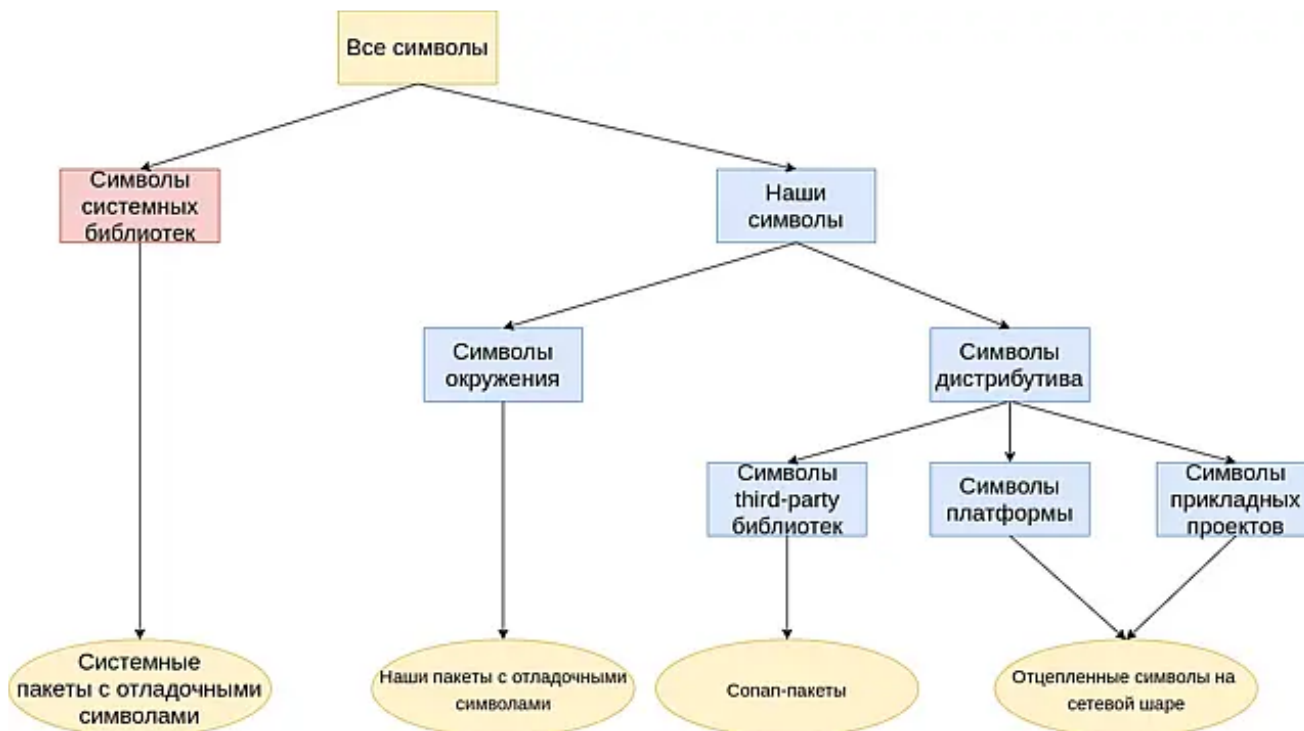
Специфика языка C++ такова, что объем отладочных данных, генерируемых во время сборки, существенно (в экстремальных случаях - в десятки раз) превышает объем собственно бинарного кода. Например, при сборке под Linux размер базовой библиотеки платформы libsbis-lib300.so составляет примерно 7 мегабайт, а вот ее отладочные символы "весят" около 21 мегабайта. И это несмотря на то, что дополнительно при сборке применяются сжатие и дедупликация символов. Соответственно, при распространении ПО символы, как правило, отцепляют, чтобы уменьшить объем дистрибутива (ну и заодно затруднить анализ программы). Так поступают мейнтейнеры всех известных автору дистрибутивов Linux, так поступаем и мы в Тензоре.

Для успешной отладки наличие символов является критически важным. Если для какого-то программного компонента, вызов кода из которого присутствует на стеке потока, отсутствуют символы, то вполне вероятна ситуация, при которой отладчик либо не сможет (полностью или частично) отобразить стек вызовов, либо вообще исказит его (т.е. начнет врать).

Также необходимо отдавать себе отчет в том, что компиляторы C/C++ весьма агрессивно оптимизируют код в процессе сборки (а дампы, как правило, поступают именно с оптимизированных сборок). Это приводит к тому, что не все абстракции (переменные, функции), которые присутствуют в исходном коде программы, имеют какое-то выражение в конечном бинарном коде. Кроме того, компилятор в ряде случаев в праве изменять порядок исполнения инструкций, если это не меняет "наблюдаемое поведение" программы. Все это может приводить к тому, что при отладке вы вместо значения интересной вам переменной увидите надпись в духе "optimized out" или "value not available". Поделаться с этим ничего нельзя.

Следует учитывать, что любой отладчик - это достаточно сложное ПО, в котором также могут быть баги или нереализованный функционал. Тот же [GDB](#) ведет свою историю из 80-х годов, что означает, что в его кодовой базе накопилось внушительное количество legacy и разного рода костылей. Соответственно, на конкретной программе/дампе отладчик может не работать в каких-то отдельных аспектах (например, не показывать переменные на каком-то фрейме стека), либо вовсе падать. В таких ситуациях надо искать workaround-ы, жаловаться на баги отладчика кому-либо в Тензоре абсолютно бессмысленно: вам смогут только искренне посочувствовать и пожелать удачи.

Теперь попробуем понять, символы от каких компонентов нам могут потребоваться при отладке.



Любое наше приложение включает бинарные файлы из трех источников:

- Непосредственно имплементирующие прикладную логику приложения (1)
- SDK платформы (2)
- Сторонние библиотеки, на которых основаны компоненты SDK платформы, либо компоненты прикладного решения (как правило - это сопак-пакеты) (3)

Но это не все. Дополнительно, любое приложение обязательно использует какие-либо компоненты из состава операционной системы, на которой оно выполняется (kernel32.dll, libc.so и т.д.) (4).

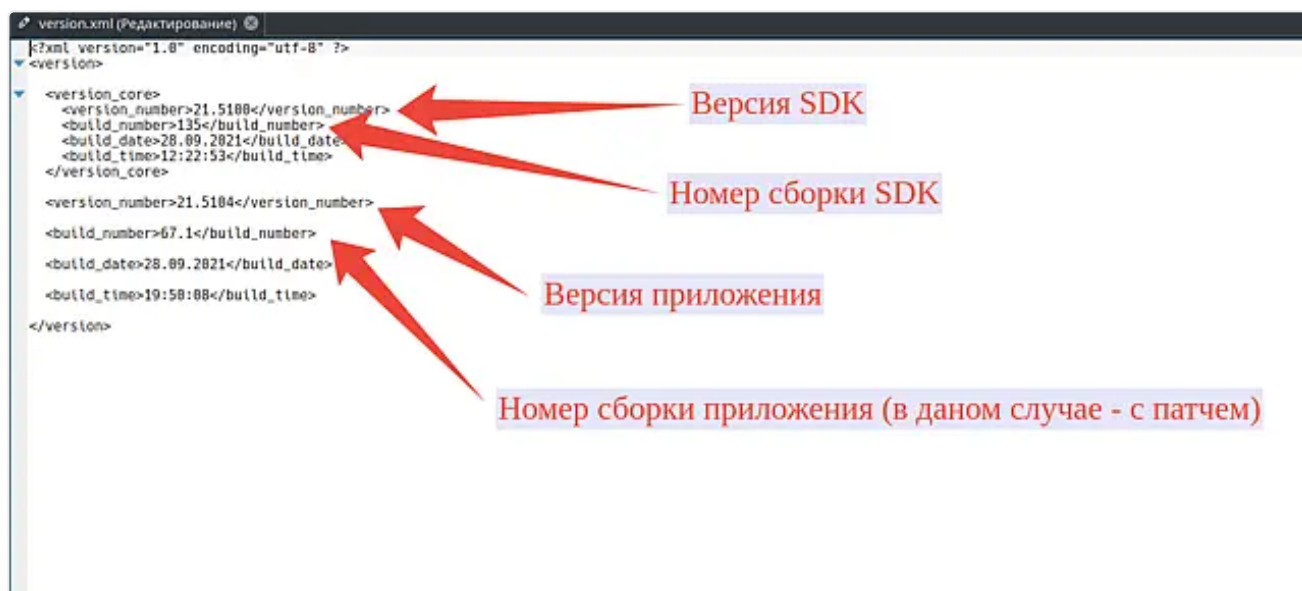
Кроме чисто системных компонентов, использоваться могут и предустановленные компоненты, являющимися прикладными с точки зрения ОС (типичный пример - интерпретатор Python-a) (5).

Соответственно, для успешной отладки дампа нужно "добыть" символы для компонентов из всех перечисленных источников (хотя как правило, символы из (3) не требуются). Процесс добычи символов является платформозависимым.

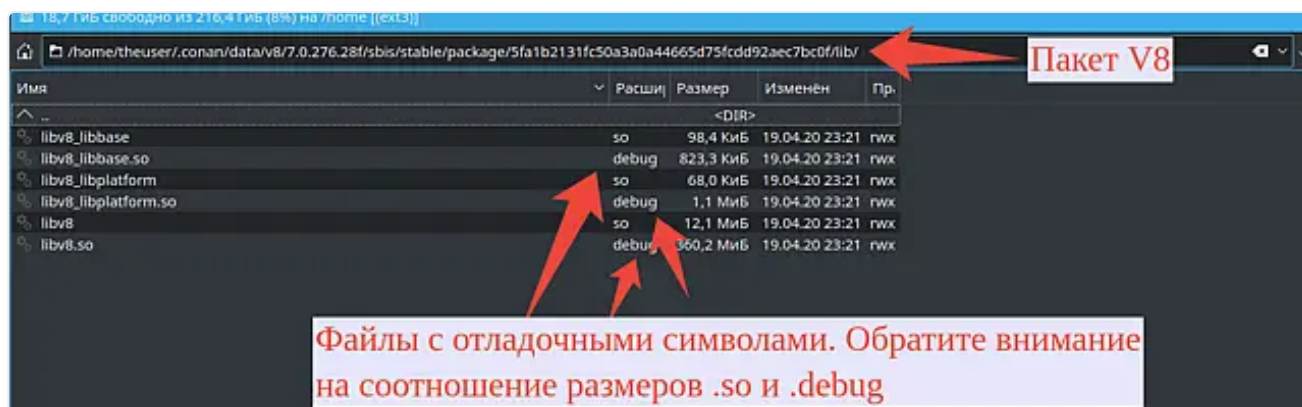
Где взять символы в Linux?

В Linux процесс получения символов является более трудоемким, чем в Windows. Особенно это касается получения символов системных библиотек. Итак, начнем по порядку.

Символы для компонентов из состава SDK-платформы и конечного прикладного продукта можно найти на специализированной сетевой шаре. На момент написания данной статьи это каталог symbols_nix на сервере [dev-symbols.unix.tensor.ru](#). На рабочих компьютерах разработчиков эта шара, как правило, примонтирована в каталог /mnt/symbols. Чтобы найти нужные символы, необходимо знать версию и номер сборки упавшего приложения, а также SDK, на котором приложение было собрано. Эту информацию можно посмотреть в файле version.xml, который присутствует в корневом каталоге всех наших приложений. Если скорость сетевого соединения до головного офиса Тензора является не очень высокой, может иметь смысл скопировать каталоги с символами с сетевой шары на жесткий диск.



Если для отладки вам необходимы символы от сторонних библиотек (например, от V8), то их можно найти в соответствующем Conan-пакете.



Что же касается символов от системных библиотек и окружения, то в Linux они распространяются в виде так называемых debuginfo-пакетов. Это специальные rpm (или deb) пакеты, содержащие отцепленные отладочные символы и исходники для основных пакетов. Как правило, они лежат в отдельных репозиториях и сами эти репозитории по умолчанию не подключены у конечных пользователей (хотя в списке репозиториях они присутствуют). После установки такого пакета отладочная информация помещается в какой-либо подкаталог каталога `/usr/lib/debug` (по крайней мере в CentOS 7). Для установки debuginfo-пакетов в CentOS 7 предусмотрена специализированная утилита под названием `debuginfo-install`. Она входит в состав пакета `yum-utils`.

В качестве необходимого окружения для Linux мы на данный момент поставляем два пакета:

- сборка стандартной библиотеки C++ - ставится в `/opt/gcc-9.1.1`.
- сборка Python 3.11 - ставится в `/opt/python3.11`.

В наших репозиториях есть соответствующие debuginfo-пакеты, они стабильны и могут успешно применяться для анализа дампов или отладки.

С "истинно" системными библиотеками ситуация хуже. Вы можете поставить локально debuginfo-пакеты, соответствующие установленным у вас локально пакетам (и это нужно сделать). Однако проблема состоит в том, что каждая виртуальная машина в Тензоре, по сути, имеет некоторый уникальный набор системных пакетов, соответствующих пакетной базе дистрибутива на момент развертывания данной конкретной VM. Массовое обновление системных пакетов в ЦОД-е компании не производится, т.к. неоднократно фиксировались прецеденты возникновения аварий после таких обновлений. Из этого следует, что, скорее всего, символы, соответствующие вашим системным пакетам, не будут в точности соответствовать системным пакетам с того сервера, где произошло падение. Однако, в 99.9% случаев это и не требуется. Для оставшегося 0.1% также существует способ решения проблемы, состоящий в закачке нужных пакетов по build-id с помощью все той же утилиты `debuginfo-install`. Однако он весьма трудоемок и в рамках данной статьи мы его рассматривать не будем.

Для того чтобы указать отладчику GDB местоположение символов, необходимо воспользоваться командой:

```

1 set debug-file-directory
  ${ПутьКСимволам1}:${ПутьКСимволам2}:...:${ПутьКСимволамN}

```

Исходя из всего вышеизложенного, в типовом сценарии отладки дампа нужно дать примерно такую команду:

```
1 set debug-file-directory
  /usr/lib/debug:${ПутьКСимволамСборкиSDK}:${ПутьКСимволамСборкиСервиса}
```

Например:

```
1 set debug-file-directory
  /usr/lib/debug:/mnt/symbols/22.1100/Platforma/linux_x86_64_clang/167:/mnt/symbols/21.1100/online-inside/45
```

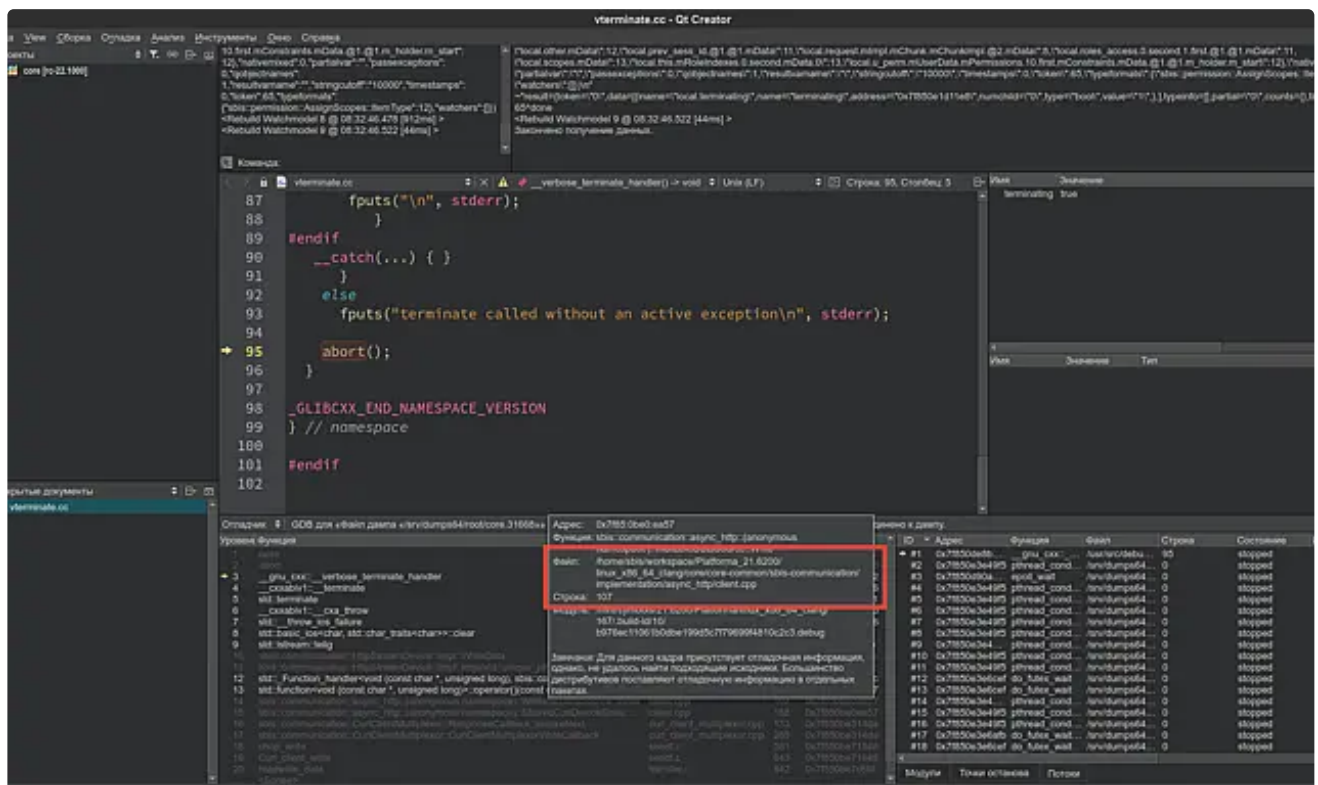
Если анализ проводится в IDE Qt Creator, то перед открытием дампа удобно бывает вписать соответствующую команду GDB в поле "Дополнительные команды запуска" на странице настроек отладчика (аналогично `set sysroot` или `set solib-search-path`).

Где взять символы в Windows?

На Windows задача получения символов полностью аналогична задаче получения `binary` и решается при помощи тех же серверов символов.

Сопоставление дампа с исходным кодом на Linux

После загрузки символов отладчиком он становится способен сопоставить стеки в дампе/отлаживаемом процессе с исходным кодом, из которого был собран дистрибутив, а также показать значения переменных. В принципе, как правило этого бывает достаточно для анализа. Но для большего удобства при работе в IDE хотелось бы, чтобы фреймы стека вызовов отображались в локальную копию исходного кода программы. Однако проблема состоит в том, что, как правило, при сборке в символах "зашиваются" абсолютные пути к исходникам в том виде, как они были на сборочном сервере.



У проводящего же анализ разработчика исходники, скорее всего, размещены по другим путям. Соответственно, отладчик/IDE просто не находит исходники на диске и не может их загрузить. Чтобы решить эту проблему, необходимо задать соответствие путей на сборочном сервере путям на компьютере разработчика. Делается это при помощи команды отладчика `set-substitute-path` вида:

```
1 set-substitute-path ${ПрефиксНаСборочномСервере}
  ${ПрефиксНаКомпьютереРазработчика}
```

Например, при сборке SDK платформы на сборочном сервере основной репозиторий платформы был выкачан по пути `/home/sbis/workspace/Platforma_22.1100/linux_x86_64_clang/core`, а локально он располагается

в /home/username/git/core. Тогда при помощи команды:

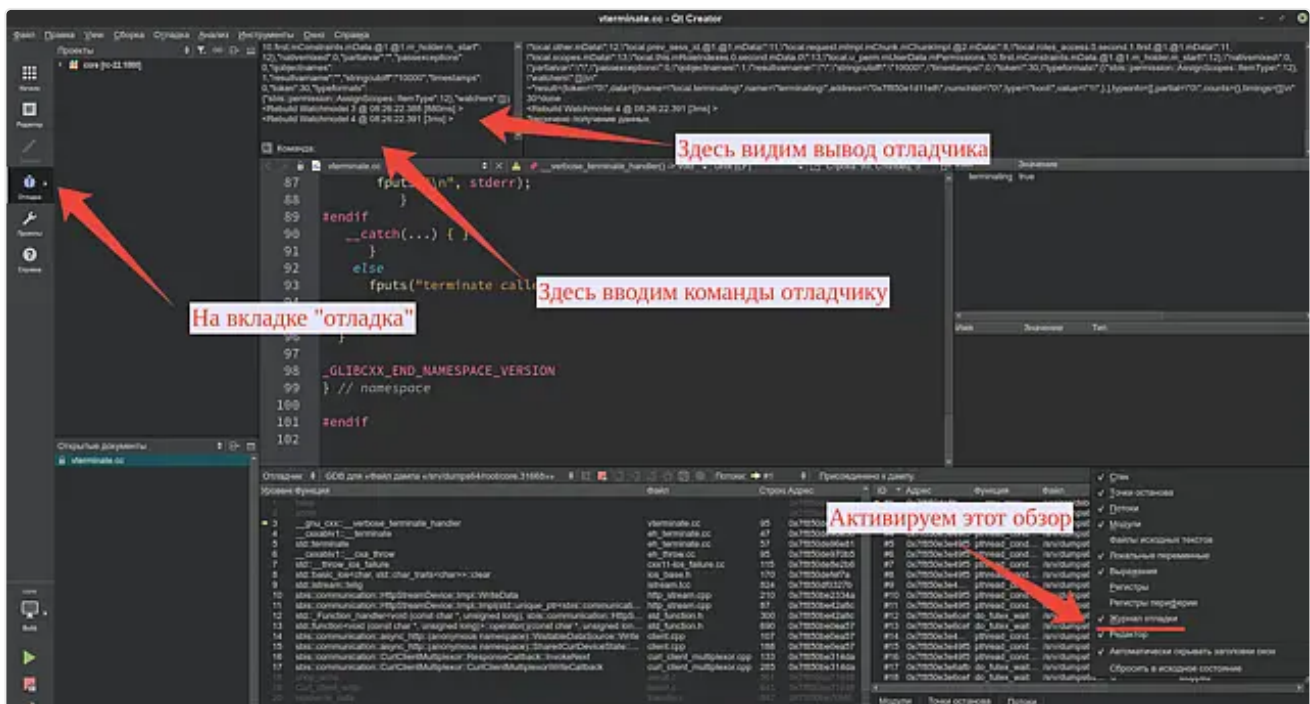
```
1 set-substitute-path
/home/sbis/workspace/Platforma_22.1100/linux_x86_64_clang/core
/home/username/git/core
```

мы можем задать сопоставление путей и при анализе использовать локальную версию исходников.

У части компонентов приложения пути в символах могут быть не абсолютными, а наоборот, относительным (например, multi.c). Касается это сторонних библиотек, собираемых собственными сборочными скриптами и интегрируемыми в наши решения в виде сонап-пакетов. В таком случае вместо задания соответствия "серверных" путей "локальным" следует воспользоваться командой `directory`, которая просто добавляет некоторый каталог к списку тех каталогов, в которых отладчик ищет исходники программы. Например,

```
1 directory /home/username/git/boost
```

При использовании IDE Qt Creator для доступа к консоли GDB/CDB во время отладки следует воспользоваться обзором "Журнал отладки".



Экспресс-анализ дампов в Linux

Как можно было заметить по описанию выше, процесс "ручной" настройки анализа дампов в Linux требует совершения ряда механических операций, отнимающих драгоценное время разработчика. При этом в большинстве случаев ошибки, приведшие к выпадению дампа, являются достаточно тривиальными и не требуют глубокого анализа стека вызовов.

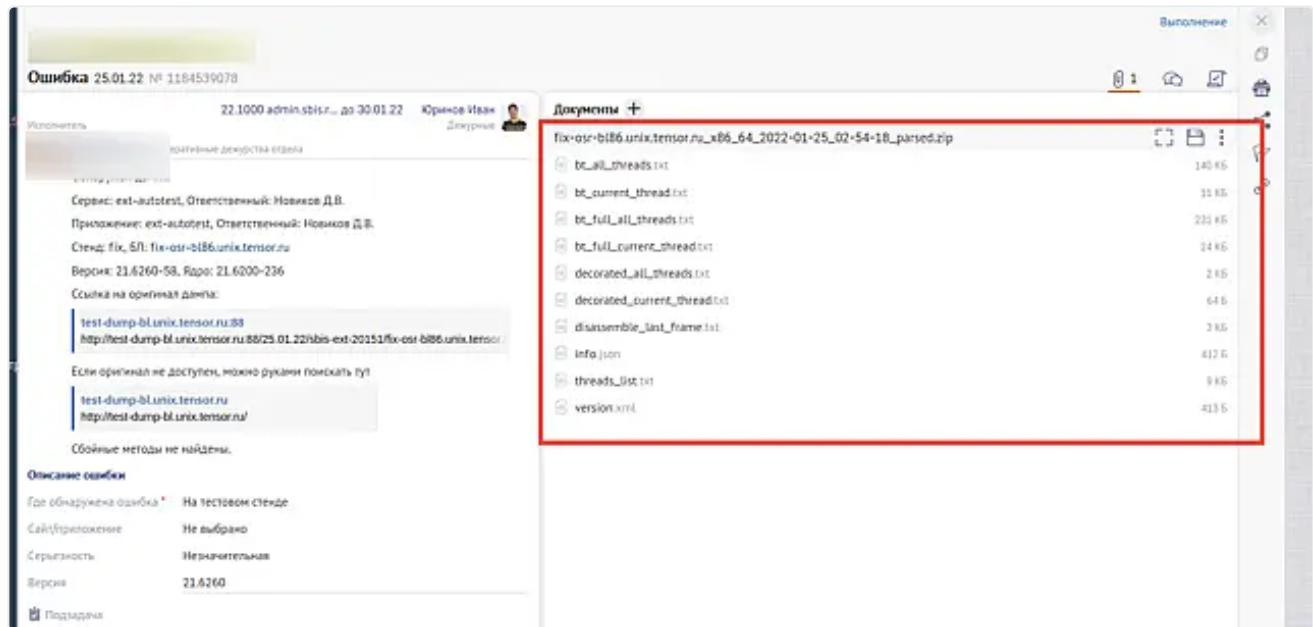
"Лидерами рейтинга" являются:

- разыменование нулевого указателя/не инициализированного optional:
нужно просто добавить проверку / сменить тип переменной.
- выброс исключения в поехсерт-контексте:
нужно не бросать исключение / обработать его.
- переполнение стека:
нужно добавить проверки, ограничивающие рекурсию.
- SIGBUS:
нужно разбираться, кто съел память на сервере и достаточно ли ее в принципе.

Для того чтобы сэкономить время, необходимое для первичного (а зачастую и окончательного) анализа, была разработана специализированная утилита, которая согласно регламенту работы с дампами в компании автоматически

исполняется для всех архивов с дампами с продуктива или тестового контура (а также ее можно попросить прогнать вручную).

Результатом работы данной утилиты являются файлы, предоставляющие основную информацию в виде текстовых файлов. Пример ошибки, к которой приложен результат работы утилиты, можно увидеть на картинке ниже.



Генерируются следующие файлы:

- info.json

В этом файле содержится вспомогательная информация к дампу: адрес сервера, где дамп был сгенерирован, pid упавшего процесса, номер сигнала, из-за которого был сгенерирован дамп и т.д. Именно с этого файла надо начинать анализ;

- version.xml

Файл, в котором записана версия упавшего приложения (как уже упоминалось выше, версия состоит из двух частей: версия и номер сборки SDK платформы и версия и номер сборки собственно приложения);

- bt_current_thread.txt

Стек потока (без значений переменных), в котором произошел сбой. Обратите внимание, что для дампов, сгенерированных вручную, тут всегда будет первый поток и признаков сбоя там не будет (т.к. никакого сбоя и не произошло);

- bt_full_current_thread.txt

То же, что bt_current_thread.txt, но со значениями локальных переменных;

- bt_all_threads.txt

Стеки всех потоков приложения без значений переменных;

- bt_full_all_threads.txt

То же, что bt_all_threads.txt, но со значениями переменных;

- decorated_current_thread.txt

Стек "сбойного" потока в терминах "Методы БЛ" и ссылки на код на Python;

- decorated_all_threads.txt

Стеки всех потоков в терминах "Методы БЛ" и ссылки на код на Python;

- disassemble_last_frame.txt

Дизассемблированный последний фрейм "сбойного" потока;

- threads_list.txt

Список потоков программы.

Следует отдавать себе отчет, что ввиду оптимизаций компилятора в файлах могут быть видны не все локальные переменные. Также следует понимать, что ввиду наличия багов в GDB бывает, что генерируются не все файлы (потому что при генерации остальных отладчик просто падает). Ну и наконец, фиксировались случаи, когда сам архив с дампом был поврежден/записан не до конца. В этом случае анализу он не поддается ни в ручном, ни в автоматическом режиме.

Ручная генерация дампов

Классом ошибок, при исследовании которых дампы можно назвать самым лучшим инструментом, является зависание приложения или его части. Можно очень долго читать длинные логи, предшествующие зависанию, или с задумчивым видом смотреть на код программы, однако только дамп может быстро дать полное представление о том, где же именно оно "висит". Соответственно, если сотрудник эксплуатации сообщает вам, что на одном из серверов продуктивного (или тестового) контура обнаружен долго работающий метод БЛ и есть подозрение на его зависание, то первое, что нужно сделать – это снять дамп с данного процесса.

В Linux следует действовать так:

- Для страховки снимаем "просто" дамп при помощи утилиты gcore:

```
sudo gcore ${PIDЗависшегоПроцесса}
```

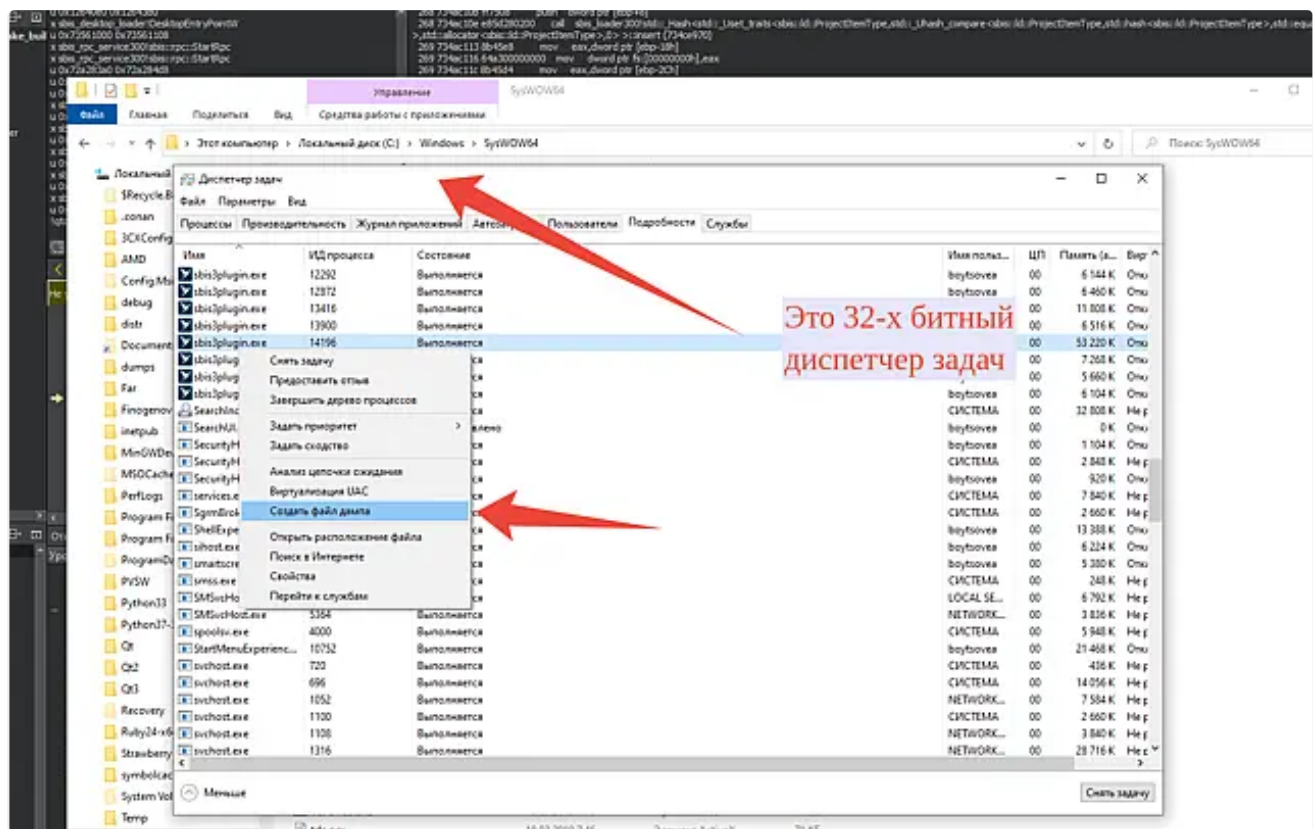
Как результат получаем только core-файл, его анализ придется осуществлять полностью вручную (в т.ч. собирать binary, символы и вот это вот все, см. выше).

- При помощи kill или htop посылаем зависшему процессу сигнал SIGABORT.

Это приведет к аварийному завершению процесса и записи дампа на уровне ОС с вызовом соответствующего обработчика, который (если все настроено правильно) сформирует архив с дампом, пригодный для анализа вышеописанной автоматизированной утилитой (ну и гораздо более приятный для "ручного" анализа, если он все-таки потребуется).

В Windows запись дампов осуществляется при помощи диспетчера задач. Однако т.к. на данный момент наши приложения под Windows являются 32-х битными, то и диспетчер задач следует использовать 32-х битный.

Запускать надо файл <C:\\Windows\\SysWOW64\\Taskmgr.exe> от имени администратора. Затем перейти на вкладку "Подробности", найти интересующий процесс по pid, выделить его, щелкнуть правой кнопкой мыши и из выпадающего меню выбрать пункт "Создать файл дампа". Запись дампа таким способом не приводит к аварийному завершению процесса.



Про ошибки "Проверьте наличие dump в каталоге сервиса"

В логах сервера приложений иногда можно встретить ошибки вида "Дочерний процесс (pool:N) (pid:M) (state: K)аварийно завершился. Проверьте наличие dump в каталоге сервиса."

Логи транзакции: операции с 23.03.2022 00:00:00, до конца времени с 19.03.2022 00:00:00						
Таблица: таблица транзакций						
Время	Описание Мод. Состояние	Ид. Внешней Внутренней	Получен. Потеря Потеря процесса [ID]	Тип Путь	Увед. Метод Ид. Выхода	Сообщение
25.01.22 10:38:13.94			26972 43036 n			Дочерний процесс [root.0] [pid:34484] (state: 0) завершился. Проверьте наличие дампа в каталоге сервиса.
25.01.22 10:38:14.00			26972 43036 n			Дочерний процесс [root.0] [pid:36068] (state: 0) завершился. Проверьте наличие дампа в каталоге сервиса.
25.01.22 10:38:14.51			26972 12676 n			Дочерний процесс [root.1] [pid:11196] (state: 0) завершился. Проверьте наличие дампа в каталоге сервиса.
25.01.22 10:38:14.55			26972 12676 n			Дочерний процесс [root.1] [pid:42656] (state: 0) завершился. Проверьте наличие дампа в каталоге сервиса.
25.01.22 10:38:14.88			26972 43036 n			Дочерний процесс [root.0] [pid:21836] (state: 0) завершился. Проверьте наличие дампа в каталоге сервиса.
25.01.22 10:38:14.90			26972 43036 n			Дочерний процесс [root.0] [pid:21652] (state: 0) завершился. Проверьте наличие дампа в каталоге сервиса.
25.01.22 10:38:15.40			26972 12676 n			Дочерний процесс [root.1] [pid:15626] (state: 0) завершился. Проверьте наличие дампа в каталоге сервиса.
25.01.22 10:38:15.43			26972 12676 n			Дочерний процесс [root.1] [pid:29736] (state: 0) завершился. Проверьте наличие дампа в каталоге сервиса.

Такого рода ошибки записываются в лог в том случае, когда управляющий процесс сервера приложений (напомню, что сервер приложений у нас в типовой конфигурации многопроцессный) фиксирует неожиданное с его точки зрения завершение одного из своих рабочих процессов. В данном случае запись про наличие дампов пишется чисто **СПЕКУЛЯТИВНО**. Мы не знаем истинной причины завершения процесса, ее установление никоим образом не является работой сервера приложений, и потому в тексте ошибки мы просто предлагаем проверить, нет ли случайно дампов с данного сервера в похожий момент времени (т.к. в большинстве случаев падение процесса все-таки сопровождается записью дампа). Однако дамп будет далеко не всегда. Например, дамп не пишется, если:

- процесс убил OOM-killer или в системе просто не хватает памяти для записи дампа (также при дефиците памяти в системе могут формироваться "битые" архивы с дампами, не поддающиеся анализу).
- процесс вышел каким-то стандартным способом (например, код какой-то сторонней библиотеки явно позвал `exit()`).
- кто-то или что-то послало процессу сигнал, обработка которого не сопровождается записью дампа (например, `SIGKILL`).

Соответственно, если запись в логах есть, а дампа нет, то необходимо обращаться к системным журналами того сервера, на котором ошибка была сгенерирована. Возможно, там вы найдете какую-то дополнительную полезную информацию.